

PARTE A — GIT COLABORATIVO (1 h sincrónica + práctica asincrónica)

A.1 Marco teórico

Ramas: la herramienta central del trabajo profesional

Una **rama (branch)** es simplemente un puntero móvil a un commit; crearla es instantáneo y no duplica archivos. La rama por defecto es **main** y en el curso representará siempre la **versión estable y entregable** del portafolio. El flujo profesional que adoptaremos se llama **feature branch workflow**: cada nueva funcionalidad o práctica se desarrolla en su propia rama; cuando está lista y revisada, se **fusiona (merge)** a main. Así, main nunca se rompe y varias personas (o varias tareas) avanzan en paralelo sin estorbarse.

Concepto	Definición operativa
main	Rama estable. Solo recibe trabajo terminado mediante fusiones.
Rama de trabajo	Copia paralela donde se desarrolla una tarea. Convención del curso: practica/sNN-tema (p. ej. practica/s02-xml).
HEAD	Puntero a la rama/commit donde se está trabajando ahora.
Fusión fast-forward	main no cambió mientras se trabajaba: Git solo mueve el puntero hacia adelante.
Fusión de tres vías (three-way)	Ambas ramas avanzaron: Git crea un commit de fusión combinando los cambios.
Conflicto	Las dos ramas modificaron LAS MISMAS LÍNEAS del MISMO archivo: Git no decide por usted; marca el archivo y una persona resuelve.

Anatomía de un conflicto — Git inserta marcadores en el archivo; resolver significa editar el archivo dejando la versión final deseada (puede ser una, otra o una combinación) y eliminar los marcadores:

```
<<<<<< HEAD
Bienvenidos a mi portafolio del INA
=====
Portafolio de evidencias – CSTI12002
>>>>>> practica/s02-titulo
```

Repositorios remotos: publicar y sincronizar

Un **remoto** es una copia del repositorio hospedada en un servidor (GitHub). Por convención, el remoto principal se llama **origin**. Cuatro operaciones cubren el ciclo completo de sincronización:

Comando	Función
git clone <url>	Descarga un repositorio remoto completo (historial incluido) a la máquina.
git push	Sube los commits locales al remoto. La primera vez: git push -u origin main (la -u recuerda el destino).
git fetch	Descarga las novedades del remoto SIN mezclarlas con el trabajo local (permite revisarlas primero).
git pull	fetch + merge: descarga y fusiona las novedades en la rama actual. Regla profesional: pull ANTES de empezar a trabajar y push AL TERMINAR.

Autenticación: GitHub ya no acepta la contraseña de la cuenta para hacer push por HTTPS. En Windows, el **Git Credential Manager** (incluido con Git) abre una ventana del navegador la primera vez y guarda la credencial de forma segura. La alternativa profesional son las **llaves SSH** (se menciona; la guía queda como material complementario) y los **Personal Access Tokens (PAT)** para casos especiales.

Pull Requests: la revisión de código profesional

Un **Pull Request (PR)** es la solicitud formal, hecha en GitHub, de fusionar una rama en otra. Es el corazón del trabajo en equipo real: permite ver el diff completo, comentar línea por línea, pedir cambios, aprobar y finalmente fusionar con evidencia de todo el proceso. En el curso: cada práctica se entrega como PR de su rama hacia main; en los trabajos en parejas, el compañero **revisa y comenta** el PR antes de la fusión (así se evalúa también la comunicación asertiva). Conceptos vecinos que se mencionan: **fork** (copia de un repositorio ajeno a la cuenta propia, base de la contribución a código abierto), **Issues** (tiquetes de tareas/errores) y **colaboradores** (personas con permiso de escritura: Settings → Collaborators).

Interfaces gráficas

- **Source Control de VS Code:** cubre el 95 % del flujo diario — stage, commit, push/pull (botón Sincronizar), creación y cambio de ramas (esquina inferior izquierda), editor visual de conflictos con botones “Aceptar cambio actual / entrante / ambos”, y gráfico de ramas.
- **GitHub Desktop** (desktop.github.com): cliente gráfico oficial, útil para quien prefiere no usar terminal; mismo Git por debajo.
- **GitHub web:** edición rápida de archivos, revisión de PRs, historial visual. Mensaje clave: la herramienta cambia, los conceptos (rama, commit, merge, remoto) son los mismos.

♦ **PROTOCOLO OFICIAL DEL CURSO (vigente desde hoy hasta la sesión 38):** 1) pull antes de trabajar · 2) rama practica/sNN-tema · 3) commits atómicos con la convención tipo: descripción · 4) push de la rama · 5) Pull Request con descripción de lo realizado · 6) revisión (propia o de par, según la práctica) · 7) merge a main · 8) en las evaluaciones, entregar repositorio_url.txt con la URL del repositorio. Este protocolo será parte de la lista de cotejo de TODAS las prácticas.

A.2 Demostración guiada (pantalla compartida; el estudiantado replica)

Bloque 1 — Ramas y fusión en local (repositorio demo-git de ayer)

1. Ver las ramas existentes: **git branch** (el asterisco marca la actual).
2. Crear y cambiarse a una rama de trabajo: **git switch -c practica/s02-titulo** (equivale al antiguo **git checkout -b**). Verificar en la esquina inferior izquierda de VS Code que ahora dice practica/s02-titulo.
3. Modificar el título del README, confirmar con feat: mejora el título del portafolio. Mostrar con **git log --oneline --all --graph** cómo la rama avanzó y main quedó atrás.
4. Volver a main (**git switch main**) y abrir el README: el cambio “desapareció” — cada rama tiene su propia versión de los archivos. Momento clave de comprensión.
5. Fusionar: **git merge practica/s02-titulo**. Señalar en la salida que fue **fast-forward**. Borrar la rama ya fusionada: **git branch -d practica/s02-titulo** (higiene profesional).

Bloque 2 — Provocar y resolver un conflicto (a propósito)

1. Crear la rama **practica/s02-lema**, editar la PRIMERA línea del README con un lema, confirmar.
2. Volver a main y editar LA MISMA línea con un texto distinto; confirmar.
3. Ejecutar **git merge practica/s02-lema** → Git reporta CONFLICT. Abrir el README: VS Code resalta el bloque con los marcadores y ofrece los botones **Aceptar cambio actual / Aceptar cambio entrante / Aceptar ambos / Comparar**.
4. Resolver combinando ambos textos manualmente, eliminar los marcadores, guardar, **git add README.md** y **git commit** (mensaje sugerido por Git: Merge branch...). Mensaje al grupo: un conflicto NO es un error, es Git pidiendo una decisión humana.

Bloque 3 — Publicar en GitHub y ciclo remoto completo

1. En GitHub: **New repository** → nombre demo-git → público → SIN inicializar con README (el repositorio ya existe en local). Copiar la URL HTTPS.

2. Conectar y publicar: ver comandos abajo. La primera vez, autenticarse en la ventana del Credential Manager.
3. Refrescar GitHub: mostrar archivos, historial de commits, y el README renderizado como portada.
4. Editar un archivo directamente en la web de GitHub (botón lápiz) y confirmar allá; en local, ejecutar **git pull** y ver llegar el cambio: queda demostrada la sincronización bidireccional (simula el trabajo de otra persona).
5. Ciclo con PR: crear rama **practica/s02-seccion-pr**, agregar una sección al README, push de la rama (**git push -u origin practica/s02-seccion-pr**); en GitHub aparece el aviso “Compare & pull request” → crear el PR con título y descripción → revisar la pestaña Files changed → **Merge pull request** → en local: **git switch main** y **git pull**.
6. Mostrar 60 segundos de **GitHub Desktop** con el mismo repositorio, para que quien lo prefiera lo explore por su cuenta.

```
git remote add origin https://github.com/USUARIO/demo-git.git
git remote -v          # verifica el remoto configurado
git push -u origin main # primera publicación

# ciclo diario a partir de ahora:
git pull                # antes de trabajar
git push                # al terminar
```

A.3 Práctica asincrónica A — “Portafolio publicado y flujo profesional completo”

1. Crear en GitHub el repositorio **portafolio-CSTI12002** (público, sin inicializar) y conectarlo al repositorio local de la Sesión 1: **git remote add origin <URL>** y **git push -u origin main**. Verificar en el navegador que se ven las 5 carpetas y el README.
2. Crear la rama **practica/s02-perfil** (**git switch -c practica/s02-perfil**).
3. En esa rama, agregar al README una sección “Protocolo de trabajo del curso” copiando los 8 pasos del protocolo oficial, y en `unidad-01-control-versiones/bitacora.md` la entrada de la sesión 2. Realizar 2 commits atómicos (docs: ...).
4. Publicar la rama (**git push -u origin practica/s02-perfil**) y abrir un **Pull Request** hacia main con una descripción de dos líneas de lo realizado.
5. **Ejercicio de conflicto controlado:** siguiendo la guía publicada en la plataforma, editar la MISMA línea de la bitácora en main (desde la web de GitHub) y en la rama local; intentar la fusión, resolver el conflicto en VS Code y confirmar la resolución.
6. Fusionar el PR en GitHub, y en local: **git switch main + git pull**. Borrar la rama local ya fusionada.
7. **Trabajo colaborativo real (en parejas asignadas):** en Settings → Collaborators, invitar al compañero; el compañero acepta, CLONA el repositorio del otro (**git clone**), crea la rama **colaboracion/sugerencia**, agrega una línea de retroalimentación amable al final del README (capacidad del Convivir), hace push y abre un PR; la persona dueña lo revisa, deja UN comentario y lo fusiona. Cada quien hace ambos roles.
8. Entrega en la plataforma: URL del repositorio + captura del PR fusionado del compañero.

Lista de cotejo — Práctica A

Criterio	✓
Portafolio publicado en GitHub con historial completo de la sesión 1	
Rama practica/s02-perfil creada, con 2 commits convencionales, publicada y fusionada vía PR	
Conflicto provocado y resuelto correctamente (sin marcadores residuales en el archivo)	
Colaborador invitado; PR del compañero revisado, comentado y fusionado	

PARTE B — FUNDAMENTOS DE LA WEB Y XML (3 horas)

B.1 Marco teórico: cómo funciona la web

Internet no es la web

Internet es la infraestructura: la red mundial de redes que interconecta dispositivos mediante el protocolo TCP/IP. La **web (World Wide Web)** es uno de los servicios que corre sobre Internet (como también lo son el correo o la transferencia de archivos): un sistema de documentos de hipertexto enlazados, creado por Tim Berners-Lee en 1989-1991, cuyas tres bases son **URL** (dónde está el recurso), **HTTP** (cómo se pide y entrega) y **HTML** (cómo se escribe el documento).

Arquitectura cliente-servidor y el viaje de una petición

La web funciona con el modelo **cliente-servidor**: el **cliente** (el navegador) solicita recursos; el **servidor** (una computadora siempre disponible que “sirve” páginas) responde. El viaje completo al escribir una dirección y presionar Enter:

1. El navegador analiza la **URL**: protocolo (https), **dominio** (www.ejemplo.com), ruta (/carpeta/pagina.html) y parámetros opcionales.
2. Consulta el **DNS** (el “directorio telefónico” de Internet) para traducir el dominio a una **dirección IP** — el identificador numérico del servidor (IPv4: 190.113.24.10; IPv6: formato extendido moderno).
3. Establece la conexión (TCP, cifrada con TLS si es https) y envía una **petición HTTP**: método **GET** (dame el recurso) con sus cabeceras. Otros métodos: POST (enviar datos, p. ej. formularios — se retomará en la Unidad 2).
4. El servidor responde con un **código de estado** y el contenido: **200** OK, **404** no encontrado, **301** redirección, **500** error del servidor.
5. El navegador recibe el HTML, solicita los recursos vinculados (CSS, imágenes, scripts) y **renderiza** la página.

Estructura interna de un navegador

Componente	Función (ejemplos actuales)
Interfaz de usuario	Barra de direcciones, pestañas, botones — todo lo que rodea a la página.
Motor de renderizado	Interpreta HTML y CSS y dibuja la página: Blink (Chrome/Edge/Opera), Gecko (Firefox), WebKit (Safari).
Capa de red	Ejecuta las peticiones HTTP/HTTPS.
Intérprete de JavaScript	Ejecuta el código JS: V8 (Chrome/Edge), SpiderMonkey (Firefox).
Almacenamiento de datos	Cookies, localStorage, caché — persistencia en el cliente.
Motor de búsqueda (searchbar)	Integración de búsqueda desde la barra de direcciones.

Editores web del mercado: Visual Studio Code (gratuito, extensible, el estándar de facto y el del curso), Sublime Text (liviano, licencia de pago), WebStorm (IDE comercial de JetBrains), Notepad++ (básico), Brackets (descontinuado — dato de cultura general). Criterios de elección: resaltado de sintaxis, autocompletado (Emmet), integración con Git, terminal integrada y ecosistema de extensiones.

B.2 Marco teórico: XML

XML (eXtensible Markup Language) es un metalenguaje de marcado, estándar de la W3C, diseñado para **almacenar y transportar datos** de forma legible por humanos y máquinas. A

diferencia de HTML —que tiene etiquetas predefinidas para PRESENTAR contenido—, en XML **las etiquetas las define quien escribe el documento** para DESCRIBIR sus datos. Usos reales: configuración de aplicaciones, canales RSS, sitemaps de sitios web, intercambio de datos entre sistemas (p. ej. facturación electrónica de Costa Rica) y... **SVG**, el formato de gráficos vectoriales que veremos en la sesión 8, es XML.

Reglas de sintaxis: documento “bien formado”

- Prólogo opcional en la primera línea: `<?xml version="1.0" encoding="UTF-8"?>`.
- **Un único elemento raíz** que contiene a todos los demás.
- Toda etiqueta que abre, **cierra** (`<precio>...</precio>`) o es de cierre propio (`<linea/>`).
- Las etiquetas se **anidan correctamente** (la última que abre es la primera que cierra) y **distinguen mayúsculas** (`<Precio> ≠ <precio>`).
- Los **atributos** siempre llevan valor entre comillas: `<producto codigo="A-101">`.
- Caracteres especiales mediante **entidades**: `<`; `>`; `&`; `"`.
- Comentarios: `<!-- texto -->`. Un documento que cumple todas las reglas está **bien formado**; si además cumple un esquema (DTD/XSD) se dice **válido** (concepto que basta mencionar).

```
<?xml version="1.0" encoding="UTF-8"?>
<catalogo>
  <!-- Catálogo de ejemplo del curso -->
  <producto codigo="A-101" disponible="si">
    <nombre>Teclado mecánico</nombre>
    <precio moneda="CRC">28500</precio>
    <categoria>Perifericos</categoria>
  </producto>
  <producto codigo="A-102" disponible="no">
    <nombre>Monitor 24 pulgadas</nombre>
    <precio moneda="CRC">95000</precio>
    <categoria>Pantallas</categoria>
  </producto>
</catalogo>
```

B.3 Demostración guiada

1. Abrir un sitio real y las **DevTools (F12)**: en la pestaña **Network**, recargar y mostrar la primera petición (método GET, estado 200, cabeceras, tiempo); buscar un recurso inexistente para provocar y mostrar un **404**. (<https://ejemplo.com>)
2. En la pestaña **Elements**, mostrar el HTML vivo que el motor de renderizado está dibujando (anticipo de la próxima sesión).
3. Hacer un **ping** a un dominio en la terminal para visualizar la traducción dominio → dirección IP.
4. En VS Code, crear **catalogo.xml** y escribir el ejemplo anterior; provocar dos errores a propósito (etiqueta sin cerrar y mayúscula inconsistente) para que el resaltado del editor los evidencie, y corregirlos explicando la regla violada.
5. Abrir el XML en el navegador para ver cómo lo interpreta como árbol de datos plegable.

B.4 Práctica asincrónica B — “Mi primer documento de datos, con el protocolo del curso”

1. En el portafolio, crear la rama **practica/s02-xml** (recordar: pull antes de empezar).
2. Crear **unidad-01-control-versiones/catalogo.xml**: un catálogo de un tema personal (películas, recetas, equipos, música) con prólogo, elemento raíz, **mínimo 5 elementos** con **al menos 2 atributos** cada uno, 3 subelementos por elemento, un comentario y una entidad (p. ej. `&`; en un nombre).

3. Verificar que el documento está bien formado: sin subrayados de error en VS Code y visualizable como árbol en el navegador.
4. Agregar a la bitácora una entrada: “el viaje de una petición HTTP” explicado en 5 líneas con sus palabras.
5. Confirmar con commits atómicos (feat: agrega catálogo XML de <tema> · docs: explica el viaje de una petición), push de la rama, **Pull Request** y fusión a main.
6. Entrega en la plataforma: URL del repositorio (el PR fusionado debe ser visible en la pestaña Pull requests → Closed).

3. CIERRE Y EVALUACIÓN — Autoevaluación en línea (10 ítems)

Formulario autocalificado en la plataforma (respuesta correcta entre paréntesis):

1. La rama por defecto y estable del repositorio se llama...
2. Comando para crear y cambiarse a una rama en un solo paso.
3. Comando que sube los commits locales al remoto.
4. Comando que descarga y fusiona las novedades del remoto.
5. ¿Qué significa el marcador <<<<<< HEAD en un archivo?
6. ¿Qué es un Pull Request?
7. ¿Qué servicio traduce un dominio a una dirección IP?
8. ¿Qué código de estado indica “recurso no encontrado”?
9. En XML, ¿cuántos elementos raíz puede haber?
10. ¿HTML y XML sirven para lo mismo?